Using MathCompile

MathCompile is a worksheet of math cells — calculator results stack beneath each input like a notebook, while LaTeX and Python outputs sit beside the input in a resizable column. Pick the format in the header: LaTeX source, computed answers, or SymPy Python code. It runs entirely in your browser and saves your worksheet automatically.

The worksheet at a glance

- **Cells** are numbered from the left gutter. Click a cell to edit it; the up and down arrow keys move between cells when the caret reaches an edge. **+ Add expression** appends a cell, and **×** deletes one. Pressing Backspace in a cell that holds only blank space deletes it too.
- **The output beside or beneath each cell** shows whatever the current output target produces for that cell — beside it for LaTeX and Python, beneath it for the calculator.
- **The header** holds the worksheet menu (switch, rename, delete, save/open files), Smart mode, the Output target picker, the dark-mode toggle, the **User guide** link (this document as a PDF), the **?** help-search button, and the Commands button (or press `Ctrl+K` / `⌘K`).
- **How do I…**, under the cell list, is a quick keystroke cheat sheet built from your pinned topics (it ships with a default set). `Ctrl+/` (or the **?** button, the *Search help* command, or the *Search all topics…* link in the section) opens a centered search over the full help catalog; **Enter**/click writes the topic's input into the active cell (topics with no single input, like *Greek letters*, pin instead), **Shift+Enter** or the **pin** button keeps a topic in the list, **unpin** removes one, **×** hides one (still findable in search, dimmed), and *restore default help topics* brings the original set back. This document is the long version.

The command palette (`Ctrl+K`) covers: insert an expression below, duplicate the current one, delete it, clear all expressions, toggle Smart mode and dark mode, switch the output target, switch worksheets or create a new one, save the worksheet to a file, and jump to any cell by number.

Typing math

Smart mode (the default)

With Smart mode on — the checkbox in the header — you rarely need a backslash. Whole words convert to symbols as you type, and a digit right after a letter becomes a subscript:

You want	Type	Renders
Definite integral	`int`	$\int$ with upper and lower bound blocks
Indefinite integral	`antid`	a bare $\int$
Indefinite double integral	`iint`	a bare $\iint$
Sum / product	`sum`, `prod`	$\sum$, $\prod$ with bound blocks
Square root	`sqrt`	$\sqrt{}$

You want	Type	Renders
Constants and Greek	`pi`, `theta`, `infty`	π, θ, ∞
Derivative operator	`derivative`	$\frac{d}{d}$ — caret lands in the denominator
Function definition	`def`	$\mathbf{def}\ f(x)$ — the def marker (see *Output*)
Solve equations	`solve`	$\mathbf{solve}$ — the solve marker
Solve an ODE	`dsolve`	$\mathbf{dsolve}$ — the dsolve marker
Solve a PDE	`pdesolve`	$\mathbf{pdesolve}$ — the pdesolve marker
Auto-subscript	`x2`	x_2

Operator names like `sin`, `lim`, and `det` stay upright and space correctly without any command.

LaTeX commands

Type `\` followed by a command name — `\frac`, `\sqrt`, `\int`, `\alpha` — and press **Enter** or **Space** to accept it. As you type, a suggestion menu lists matching commands; the arrow keys highlight and Enter accepts the highlight, or just keep typing to filter. `Esc` dismisses the menu. Word-runs of two or more letters suggest too (`sq` offers `\sqrt`).

`Ctrl+Space` opens the searchable symbol picker: type to filter, Enter inserts, `Esc` closes.

Unicode characters insert directly — typing `α`, `≤`, `∫`, `∀` just works.

Common things you'll type

You want	Type	Renders
Fraction	`1/2`, or `\frac` then fill the blocks	$\frac{1}{2}$
Subscript / superscript	`_`, `^` (or `x2` in Smart mode)	x_1, x^2
Square / nth root	`sqrt`, or `\nthroot`	$\sqrt{x}$, $\sqrt[3]{x}$
Sum / product with bounds	`sum` or `prod`, fill the bound blocks	$\sum_{k=1}^{n}$, $\prod_{k=1}^{n}$
Definite integral	`int`, fill the bound blocks	$\int_0^1$
Indefinite integral	`antid` or `iint` — add bounds later with `_`	$\int$, $\iint$
Limit	`\lim`, then `_{x\to0}`	$\lim_{x \to 0}$
Binomial coefficient	`\binom`	$\binom{n}{k}$
Matrix	`\pmatrix` — Enter adds a row, `Shift+Space` a column	$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$
Piecewise	`\cases`	$\begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$
Greek letters	`\alpha`, `\beta`, … or `pi`, `theta`, `infty`	α, β, π
Text inside math	`\text{…}`	rate

You want	Type	Renders
Accents	`\dot`, `\ddot`, `\bar`, `\hat`, `\vec`, `\tilde`	$\dot{x}$, $\hat{x}$, $\vec{v}$
Sets	`\in`, `\notin`, `\subseteq`, `\cup`, `\cap`, `\emptyset`, `\reals`	$x \in A$, $A \cup B$, $\varnothing$, $\mathbb{R}$
Relations	`\leq`, `\geq`, `\neq`, `\approx`, `\pm`	$a \leq b$, $a \neq b$, $a \pm b$
Arrows	`\to`, `\Rightarrow`, `\Leftrightarrow`	$x \to 0$, $p \Rightarrow q$, $p \Leftrightarrow q$
Calculus	`\partial`, `\nabla`, `\infty`, `\prime`	∂, ∇, ∞, f'
Logic	`\forall`, `\exists`, `\therefore`, `\land`, `\lor`, `\lnot`	$\forall x$, $\exists x$, $\therefore$, $\neg p$

`SYMBOLS.md` in the repository lists every command the editor accepts.

Getting in and out of blocks

Commands like `\frac` open placeholder blocks, and the caret stays inside the block you're filling. Everything you type lands there — including `+`, `=`, and spaces. To move on:

- **Right** or **Tab** steps out of the current block.
- `(` `[` `{` insert closed pairs with the caret inside; typing the closer again hops over it. `\langle` pairs with `\rangle`.
- Backspace clears the block you're in before it removes the command that owns it.

Multi-line cells and cells

Enter starts a new line inside the same cell (useful for aligned derivations); **Shift+Enter** starts a whole new cell. While a `\command` input is still open, Enter accepts the command instead of breaking a line.

Linear algebra

Environments open two ways: type `\name` directly (the shortcut) or `\begin{name}` the long way. In the table, each entry shows the shortcut and what a `a,b / c,d` grid renders as.

Matrices

You want	Type	Renders
Parenthesized matrix	`\pmatrix`	$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$
Bracketed matrix	`\bmatrix`	$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$
Braced matrix	`\Bmatrix`	$\begin{Bmatrix} a & b \\ c & d \end{Bmatrix}$
Barred matrix	`\vmatrix`	$\begin{vmatrix} a & b \\ c & d \end{vmatrix}$

You want	Type	Renders
Double-barred matrix	`\Vmatrix`	$\begin{Vmatrix} a & b \\ c & d \end{Vmatrix}$
Bare grid	`\matrix`	$\begin{matrix} a & b \\ c & d \end{matrix}$
Compact inline matrix	`\smallmatrix`	$\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}$

Inside a matrix grid: **Enter** adds a row, **Shift+Space** adds a column, and the arrow keys walk cell to cell. `&` separates columns and `\\` ends a row in the stored LaTeX. The bare forms `\pmatrix{a&b\\c&d}` and friends also work — one brace argument, no `\begin`.

Matrix operations

`det` is an upright name (works with or without Smart mode); the rest of the ops go through `\mathrm{name}` — the compiler reads upright names as functions, so these become real matrix methods in the Calculator and Python targets. A name that isn't built in, like `\rank`, types as plain `\text{rank}` — not an operator.

Operation	Type	Renders
Determinant	`det` then the matrix	$\det A$
Trace	`\mathrm{trace}` (or `\mathrm{tr}`)	$\mathrm{trace}(A)$
Rank	`\mathrm{rank}`	$\mathrm{rank}(A)$
Inverse	`A^{-1}` or `\mathrm{inverse}`	A^{-1}
Transpose	`A^{\mathrm{T}}` or `\mathrm{transpose}`	A^{T}
Norm	`\lVert` (auto-pairs)	$\lVert A \rVert$
Eigenvalues	`\mathrm{eigenvals}`	$\mathrm{eigenvals}(A)$
Eigenvectors	`\mathrm{eigenvects}`	$\mathrm{eigenvects}(A)$
Kernel / null space	`\ker`	$\ker A$
Reduced row echelon form	`\mathrm{rref}`	$\mathrm{rref}(A)$
Characteristic polynomial	`\mathrm{charpoly}`	$\mathrm{charpoly}(A)$
Null-space basis	`\mathrm{nullspace}`	$\mathrm{nullspace}(A)$
Column-space basis	`\mathrm{columnspace}`	$\mathrm{columnspace}(A)$
Diagonalization	`\mathrm{diagonalize}`	$\mathrm{diagonalize}(A)$
Adjugate	`\mathrm{adjugate}`	$\mathrm{adjugate}(A)$
Matrix exponential	`e^{A}`	e^{A}
Cross product	`\times` between vectors	$u \times v$
Dot product	`u \cdot v`	$u \cdot v$
Divergence	`\nabla \cdot` then a vector field	$\nabla \cdot F$
Elementwise product	`A \odot B`	$A \odot B$

Operation	Type	Renders
Kronecker product	`A \otimes B`	$A \otimes B$
Direct sum (block diagonal)	`A \oplus B`	$A \oplus B$

The infix ops type themselves by operand: on scalars `\cdot`, `\times`, `\odot`, and `\otimes` are ordinary multiplication, on sets they emit the Cartesian product, and `\oplus` on sets is the symmetric difference. `\nabla \cdot F` and `\nabla \times F` differentiate a concrete vector field (`[x^2, y, z]`); on an opaque name they flag rather than guess. A symbolic vector operand is declared with `u \in \mathbb{R}^{3\times1}`; a list- or matrix-bound name already counts as one.

Multi-line equations

You want	Type	Renders
Centered stack of equations	`\gathered`	$x + y = 1$ $x - y = 3$
Equations aligned on `=`	`\aligned`	$x = y$ $a = b$
Piecewise function	`\cases`	$\begin{cases} x^2 & x > 0 \\ 0 & x \le 0 \end{cases}$

Aliases open the same grid: `\gather`, `\equation`, `\multline`, and `\flalign` all open a centered stack like `\gathered`; `\align` and `\split` open an aligned grid. `\begin{…}` additionally accepts `eqnarray` and `eqalign` (also aligned grids) — those two have no `\name` shortcut. Rows and cells edit like matrices: Enter adds a row, `&` separates columns.

Custom grids

Grids that take a column spec or a count open a pending `{ }` block when you type the `\name` — enter the argument (`cc`, `l|r`, `2`) and press `}`, `Enter`, or `Tab` to apply it.

You want	Type	Renders
Grid with a column spec	`\array{lr}`	$\begin{array}{lr} a & b \\ cc & d \end{array}$
Spec'd grid for text	`\tabular{ll}`	a spec'd grid like `\array`, for text content
Stacked limit labels	`\subarray{c}`	$\begin{subarray}{c} a \\ b \end{subarray}$
Column pairs aligned together	`\alignat{2}` / `\alignedat{2}`	$\begin{aligned} x &= y & u &= v \\ a &= b & c &= d \end{aligned}$
Compact vertical stack	`\substack`	$\begin{subarray}{c} a \\ b \end{subarray}$

Set theory

The named sets have both a letter command and a word command (several spellings too — `\reals`/`\Reals`, `\integers`/`\Integers`, `\complex`/`\complexes`/`\complexplane`, `\rationals`, `\naturals`, `\quaternions`, `\primes`). Spelling out `\mathbb{R}` doesn't resolve as a command; use one of these instead.

You want	Type	Renders
Real numbers	`\R` or `\reals`	$\mathbb{R}$
Integers	`\Z` or `\integers`	$\mathbb{Z}$
Rational numbers	`\Q` or `\rationals`	$\mathbb{Q}$
Complex numbers	`\C` or `\complexes`	$\mathbb{C}$
Natural numbers	`\N` or `\naturals`	$\mathbb{N}$
Quaternions	`\H` or `\quaternions`	$\mathbb{H}$
Primes / projective	`\P` or `\primes`	$\mathbb{P}$
Integers mod n	`\Z` or `\integers`, then `_n`	$\mathbb{Z}_n$
Membership	`x` `\in` `A`, `\notin`	$x \in A$, $x \notin A$
Union / intersection / difference	`\cup` `\cap` `\setminus`	$A \cup B$, $A \cap B$, $A \setminus B$
Subset	`\subset` `\subseteq` `\nsubseteq`	$A \subset B$, $A \subseteq B$, $A \nsubseteq B$
Empty set	`\emptyset` or `\varnothing`	$\varnothing$
Finite set	`\{1,2,3\}`	$\{1, 2, 3\}$ — braces auto-pair
Set-builder	`\{ x \in \R \mid x > 0 \}`	$\{x \in \mathbb{R} \mid x > 0\}$
Indexed union / intersection	`\bigcup` `\bigcap`, then `_{i=1}^{n}`	$\bigcup_{i=1}^{n} A_i$, $\bigcap_{i=1}^{n} A_i$
Interval	`(a,b)`, `[a,b)`, `[a,b]`	(a, b), $[a, b)$, $[a, b]$ — parens/brackets auto-pair

These compile to real SymPy sets: `x \in \mathbb{R}` puts `real=True` on `x`'s symbol (likewise `\Z` → integer, `\Q` → rational, `\C` → complex), `\cup` / `\cap` / `\setminus` become `Union` / `Intersection` / `Complement`, `\{1,2,3\}` a `FiniteSet`, `\{x \mid p(x)\}` a `ConditionSet`, and `\{f(x) \mid x \in D\}` an `ImageSet`. `A \subseteq B` needs concrete sets on both sides — SymPy can't name an unknown set, so bare symbols get an `!` error note.

Applying a function to a set gives the image: `f(\{1,2,3\})` evaluates to `\{f(1), f(2), f(3)\}` (and `\{1,4,9\}` when `f` is defined). Other arguments stay fixed — `g(\{1,2\}, y)` maps over the set with `y` free — and several set arguments map over their Cartesian product.

Logic

You want	Type	Renders
Connectives	`\land` `\lor` `\neg`	$p \land q$, $p \lor q$, $\neg p$
Implication / equivalence	`\implies` `\iff`	$p \implies q$, $p \iff q$
Quantifiers	`\forall` `\exists`	$\forall x \in S$, $\exists x \in S$
Turnstile / satisfaction	`\vdash` `\models`	$T \vdash p$, $M \models p$
True / false	`\top` `\bot`	$\top$, $\bot$

You want	Type	Renders
Therefore / because	`\therefore` `\because`	$\therefore \because$

Connectives lower to `And` / `Or` / `Not` / `Implies` / `Equivalent`. A quantifier over a concrete set folds elementwise (`\forall x \in \{1,2,3\}` becomes an `And` over the elements); SymPy has no general quantifiers, so an abstract `\forall` is flagged rather than computed.

Calculus

`\int`, `\sum`, and `\prod` come with `_` / `^` bound blocks already made — type the bound, then `Right` or `Tab` to the next block. A typed `_` / `^` inside a bound nests a block instead of moving. `\lim` has no pre-made bound: type `_{x \to 0}` yourself.

You want	Type	Renders
Definite integral	`\int`, bounds `0` then `1`, `x^2` `Right` `dx`	$\int_0^1 x^2 \, dx$
Indefinite integral	`\antid`	$\int f \, dx$ — adds $+C$ in the calculator
Double / triple integral	`\iint` `\iiint`	$\iint$, $\iiint$ — boundless signs
Closed integral	`\oint` `\oiint`	$\oint$, $\oint\oint$
Derivative	`\frac{d}{dx}` or `\derivative`	$\frac{dy}{dx}$
Prime / nth derivative	`f'`, `f''`, `f^{(n)}`	$f'(x)$, $f''(x)$, $f^{(n)}(x)$
Sum / product	`\sum` then `k=1` `Right` `n`; `\prod` same	$\sum_{k=1}^{n} a_k$, $\prod_{k=1}^{n} a_k$
Limit, incl. one-sided	`\lim` then `_{x \to 0}`; `0^{+}`	$\lim_{x \to 0} f(x)$, $\lim_{x \to 0^+} f(x)$
Lim sup / lim inf	`limsup`, `liminf`	$\limsup$, $\liminf$ — upright operator names
Infinity	`\infty`	∞

Integrals, derivatives, sums, products, and limits all compile to the real SymPy calls — `integrate`, `diff`, `Sum` / `Product`, `limit` (one-sided included).

Multivariable calculus

You want	Type	Renders
Partial symbol	`\partial`	∂
Partial derivative	`\frac{\partial u}{\partial t}`	$\frac{\partial u}{\partial t}$
Partial derivative, shorter	`\partial_` `x` then `u`	$\partial_x u$ — reads as the same derivative
Iterated integral	`\iint`, `\iiint`, then `f` `dx dy`	$\iint f \, dx \, dy$, $\iiint f \, dx \, dy \, dz$
Closed surface integral	`\oiint`	$\oint\oint$
Gradient / Laplacian notation	`\nabla`	∇f, $\nabla^2 u$
Laplacian symbol	`\Delta`	Δu
Vector	`\vec{v}`	$\vec{v}$

`\partial_{x}u` compiles to the partial derivative like the `\frac` form. `\nabla f` is the gradient over the canonical coordinates (x, y, z): a bare name reads as a field of all three, `\nabla x` gives (1, 0, 0), and a coordinate bound to a value contributes a 0 in its slot. `\nabla \cdot F` compiles to divergence on a concrete vector field; `\Delta` stays display notation.

Topology

You want	Type	Renders
Map between spaces	`f : X` `\to` `Y`	$f : X \to Y$
Element mapping	`\mapsto`	$x \mapsto x^2$
Homeomorphic	`\cong`	$X \cong Y$
Product space	`\times`	$X \times Y$
Closure	`\bar{A}` or `\overline{A}`	$\overline{A}$
Boundary	`\partial` `A`	∂A
Interior	`A^{\circ}`	A°
Complement	`A'`	A'
Open / half-open sets	`(a,b)`, `[a,b)`	(a, b), $[a, b)$
Neighborhood eps / delta	`\varepsilon` `\delta`	ε, δ

Topology rows are notation — unions, intersections, closures, and maps display but carry no topological semantics. One exception to know: `\cong` is read by the compiler as modular congruence (`x \cong b \pmod m` → `Eq(Mod(x, m), b)`), not isomorphism.

Complex analysis

You want	Type	Renders
Complex numbers	`\C` or `\complexes`	$\mathbb{C}$
Real / imaginary part	`\Re` `\Im`	$\Re z$, $\Im z$
Argument	`\arg`	$\arg z$
Conjugate	`\bar{z}` or `\overline{z}`	$\overline{z}$
Modulus	`|z|`	$\|z\|$ — `|` auto-pairs
Polar form	`e^{i\theta}`	$e^{i\theta}$
Contour integral	`\oint`	$\oint_\gamma f(z)\,dz$
Weierstrass p / script ell	`\wp` `\ell`	$\wp$, ℓ

`\Re`, `\Im`, `\arg`, and `\overline{z}` compile to `re`, `im`, `arg`, and `conjugate`; `i` is the imaginary unit.

Abstract algebra

You want	Type	Renders
Integers mod n	`\Z` or `\integers`, then `_n`	$\mathbb{Z}_n$
Generated subgroup	`\langle a \rangle`	$\langle a \rangle$ — auto-pairs
Product / direct sum / tensor	`\times` `\oplus` `\otimes`	$G \times H$, $G \oplus H$, $G \otimes H$
Composition	`\circ`	$f \circ g$
Generic operation	`\star`	$a \star b$
Kernel of a map	`\ker` `\phi`	$\ker \phi$
Group order	`|G|`	$\|G\|$
Normal subgroup	`\trianglelefteq`	$N \trianglelefteq G$
Quotient	`G/H`	G/H
Lie algebra	`\mathfrak{g}`	$\mathfrak{g}$
Congruence mod m	`x \equiv b \pmod{m}`	$x \equiv b \pmod{m}$

The modular form compiles: `x \equiv b \pmod{m}` lowers to `Eq(Mod(x, m), b)`. Group notation (`\langle`, `\circ`, `\star`, `\trianglelefteq`) is display-level — infix symbols, no algebraic structure attached.

Differential equations

You want	Type	Renders
First / second derivative	`y'`, `y''`	y', y''
Time derivative	`\dot{x}` `\ddot{x}`	$\dot{x}$, $\ddot{x}$
nth derivative	`f^{(n)}(x)`	$f^{(n)}(x)$
Leibniz derivative	`\frac{dy}{dx}`	$\frac{dy}{dx}$
Dependent variable	`\text{def} y(t)`	declares `y` a function of `t`
Laplace transform	`\mathcal{L}`	$\mathcal{L}\{f\}$ — notation only

Declaring `\text{def} y(t)` marks `y` as a dependent variable, so `y'`, `y''`, and `y^{(n)}` emit real `Derivative` / `diff` calls in `t`. Dotted names read as functions of `t` on their own — `\dot{x}` is `x(t)` differentiated once. `\mathcal{L}` is notation only.

Partial differential equations

You want	Type	Renders
Partial derivative	`\frac{\partial u}{\partial t}`	$\frac{\partial u}{\partial t}$
Partial derivative, shorter	`\partial_` `t` then `u`	$\partial_t u$
Laplacian	`\nabla^{2}` `u` or `\Delta` `u`	$\nabla^2 u$, Δu
Divergence / curl	`\nabla` `\cdot` / `\nabla` `\times`	$\nabla \cdot \vec{F}$, $\nabla \times \vec{F}$
Subscript notation	`u_t`, `u_{xx}`	u_t, u_{xx}

You want	Type	Renders
Condition at infinity	`\infty`	∞

`\partial_{t}u` compiles to the partial derivative, like the `\frac` form. `\nabla u` is the gradient over (x, y, z), and `\nabla \cdot F` / `\nabla \times F` compute divergence and curl on concrete vector fields; `\Delta` stays display notation.

Solving equations

A statement that opens with `\text{solve}`, `\text{dsolve}` (or `\text{ode}`), or `\text{pdesolve}` (or `\text{pde}`) compiles to the matching SymPy solver — type `solve`, `dsolve`, `pdesolve` in Smart mode, or `\solve`, `\dsolve`, `\pdesolve`, `\ode`, `\pde` as commands.

You write	Emits
`\text{solve} x^2 = 1`	`sp.solve(sp.Eq(x**2, 1))`
`\text{solve} x^2 = 1, x`	`sp.solve(sp.Eq(x**2, 1), x)` — trailing symbols name the variables
`\text{solve} x+y=3, x-y=1, x, y`	`sp.solve([…], [x, y])` — a comma list is a system
`\text{solve} x^2 = 1 \text{ for } x>0`	`sp.solveset(…, domain=sp.Interval.open(0, sp.oo))`
`\text{dsolve} \frac{dy}{dx} = y`	`sp.dsolve(sp.Eq(sp.diff(y(x), x), y(x)), y(x))`
`\text{dsolve} … , y(0)=1`	`ics={y(0): 1}` — `y(0)=1` / `y'(0)=0` are initial conditions
`\text{pdesolve} u_x + u_t = 0`	`sp.pdsolve(…, u(x, t))`

- The unknown is inferred from the derivative — `\frac{dy}{dx}` makes `y` a function of `x` throughout the statement (a bare `y` emits `y(x)`). A `u(x,t)` spec declares the signature explicitly.
- `y'` / `y''` work inside a dsolve equation (default variable `x`), and `u_x` / `u_{xx}` / `u_{xt}` subscripts are partial derivatives.
- A `\text{dsolve}` comma list is an ODE system (`sp.dsolve([eqs], [y(x), z(x)])`); `\text{pdesolve}` takes a single equation — SymPy's `pdsolve` is first-order only, so higher-order PDEs like the heat equation come back as a SymPy `NotImplementedError` rather than a wrong answer.

Combinatorics

You want	Type	Renders
Binomial coefficient	`\binom{n}{k}`	$\binom{n}{k}$ — `binomial(n, k)`
Factorial / double factorial	`n!`, `n!!`	$n!$, $n!!$
Floor / ceiling	`\lfloor x \rfloor`, `\lceil x \rceil`	$\lfloor x \rfloor$, $\lceil x \rceil$
Sum / product	`\sum` / `\prod` with bounds	$\sum_{k=0}^{n}$, $\prod_{k=1}^{n}$
Gamma function	`\Gamma(z)`	$\Gamma(z)$ — `gamma(z)`
Totient and friends	`\mathrm{totient}(n)`	$\mathrm{totient}(n)$

You want	Type	Renders
Ellipsis	`\dots` `\cdots`	$1, 2, \dots, n$

`\binom`, `!` / `!!`, floors, ceilings, and `\Gamma` compile to the matching SymPy calls. Other number-theory words work like `totient` via `\mathrm{name}` — `mobius`, `nextprime`, `factorint` — as long as the name is a plain word.

Statistics

The calculator speaks a set of `scipy.stats` calls, typed as upright words `\mathrm{name}(args)`. The first stats cell in a session loads the scipy engine on top of SymPy (about a second more before the row lands); cells without stats names aren't affected.

Distributions

A distribution call is `<name><method>` — `normcdf`, `binompmf`, `tppf`. Available distributions: `norm`, `t`, `chi2`, `expon`, `uniform`, `beta`, `gamma`, `lognorm`, `cauchy`, `f` (continuous) and `binom`, `nbinom`, `poisson`, `geom`, `hypergeom` (discrete). Methods: `pdf` / `pmf`, `cdf`, `ppf`, `sf`, `isf`, `mean`, `var`, `std`, `median`, `stats`, `moment`, `interval`, `rvs`, and `fit` (continuous only). Distribution parameters come after the point and can be left off to get scipy's defaults — `\mathrm{normcdf}(x)` is the standard normal.

You want	Type	Renders
Density of N(0,1) at x	`\mathrm{normpdf}(x)`	$\mathrm{normpdf}(x)$ → `st.norm.pdf(x)`
Normal CDF, mean μ, sd σ	`\mathrm{normcdf}(x,\mu,\sigma)`	$\mathrm{normcdf}(x,\mu,\sigma)$ → `st.norm.cdf(x, mu, sigma)`
Inverse CDF (quantile)	`\mathrm{tppf}(0.975,9)`	$\mathrm{tppf}(0.975,9)$ → `st.t.ppf(0.975, 9)`
Upper-tail probability	`\mathrm{normsf}(2)`	$\mathrm{normsf}(2)$ → `st.norm.sf(2)`
Dist. mean / var / std	`\mathrm{binomvar}(10,0.5)`	$\mathrm{binomvar}(10,0.5)$ → `st.binom.var(10, 0.5)`
Dist. central interval	`\mathrm{tinterval}(0.95,9)`	$\mathrm{tinterval}(0.95,9)$ → `st.t.interval(0.95, 9)`
Random draws	`\mathrm{normrvs}(5)`	$\mathrm{normrvs}(5)$ → `st.norm.rvs(size=5)`
Fit a dist. to data	`\mathrm{normfit}([1,2,3,4])`	$\mathrm{normfit}(\dots)$ → `st.norm.fit(list(...))`

Sample statistics

The data argument is a list `[1,2,3]`, a set `{1,2,3}`, or a name the cell already bound (`d = [1,2,3]` then `\mathrm{ttest}(d,0)`).

You want	Type	Renders
Sample mean	`\mathrm{smean}(d)`	$\mathrm{smean}(d)$ → `np.mean(list(d))`
Median	`\mathrm{median}(d)`	$\mathrm{median}(d)$ → `np.median(list(d))`
Sample variance / sd	`\mathrm{svar}(d)`, `\mathrm{sstd}(d)`	$\mathrm{svar}(d)$ → `np.var(list(d), ddof=1)`
Std. error of the mean	`\mathrm{sem}(d)`	$\mathrm{sem}(d)$ → `st.sem(list(d))`

You want	Type	Renders
Skewness / kurtosis	`\mathrm{skew}(d)`, `\mathrm{kurtosis}(d)`	$\mathrm{skew}(d)$ → `st.skew(list(d))`
Interquartile range	`\mathrm{iqr}(d)`	$\mathrm{iqr}(d)$ → `st.iqr(list(d))`
Geometric / harmonic mean	`\mathrm{gmean}(d)`, `\mathrm{hmean}(d)`	$\mathrm{gmean}(d)$ → `st.gmean(list(d))`
Summary tuple	`\mathrm{describe}(d)`	$\mathrm{describe}(d)$ → `st.describe(list(d))`
Z-scores	`\mathrm{zscore}(d)`	$\mathrm{zscore}(d)$ → `st.zscore(list(d))`
One-sample t-test	`\mathrm{ttest}(d,0)`	$\mathrm{ttest}(d,0)$ → `st.ttest_1samp(list(d), 0)`
Conf. interval of mean	`\mathrm{meanconf}(d)` / `(d,0.99)`	$\mathrm{meanconf}(d)$ → `mc_mean_ci(d, 0.95)`

Comparing samples and hypothesis tests

Two-sample tests take two data operands; the multi-sample tests take two or more (`friedman` needs at least three). Everything prints as a `(statistic, p-value)` tuple unless noted.

You want	Type	Renders
Independent-samples t-test	`\mathrm{ttestind}(d,e)`	$\mathrm{ttestind}(d,e)$ → `st.ttest_ind(list(d), list(e))`
Paired t-test	`\mathrm{ttestrel}(d,e)`	$\mathrm{ttestrel}(d,e)$ → `st.ttest_rel(list(d), list(e))`
Mann-Whitney U	`\mathrm{mannwhitneyu}(d,e)`	$\mathrm{mannwhitneyu}(d,e)$ → `st.mannwhitneyu(list(d), list(e))`
Wilcoxon signed-rank	`\mathrm{wilcoxon}(d,e)`	$\mathrm{wilcoxon}(d,e)$ → `st.wilcoxon(list(d), list(e))`
Two-sample KS	`\mathrm{ks2samp}(d,e)`	$\mathrm{ks2samp}(d,e)$ → `st.ks_2samp(list(d), list(e))`
Correlations	`\mathrm{pearsonr}` / `\mathrm{spearmanr}` / `\mathrm{kendalltau}` `(x,y)`	$\mathrm{pearsonr}(x,y)$ → `st.pearsonr(list(x), list(y))`
Equal-variance tests	`\mathrm{levene}` / `\mathrm{bartlett}` / `\mathrm{fligner}` `(d,e)`	$\mathrm{levene}(d,e)$ → `st.levene(list(d), list(e))`
One-way ANOVA	`\mathrm{foneway}(d,e,...)`	$\mathrm{foneway}(d,e)$ → `st.f_oneway(list(d), list(e))`
Friedman test	`\mathrm{friedman}(d,e,f)`	$\mathrm{friedman}(d,e,f)$ → `st.friedmanchisquare(...)`
Normality tests	`\mathrm{shapiro}` / `\mathrm{normaltest}` / `\mathrm{jarquebera}` / `\mathrm{skewtest}` / `\mathrm{kurtosistest}` `(d)`	$\mathrm{shapiro}(d)$ → `st.shapiro(list(d))`

You want	Type	Renders
Fit against a distribution	`\mathrm{kstest}(d)` / `\mathrm{anderson}(d)`	$\mathrm{kstest}(d)$ → `st.kstest(list(d), 'norm')`
Chi-square goodness of fit	`\mathrm{chisquare}(obs,exp?)`	$\mathrm{chisquare}(obs)$ → `st.chisquare(list(obs))`
Binomial test	`\mathrm{binomtest}(k,n,p?)`	$\mathrm{binomtest}(3, 10, 0.5)$ → `st.binomtest(3, 10, 0.5)`

Worth knowing:

- `\mathrm{smean}` rather than `\mathrm{mean}` — `mean(x)` already prints the expectation symbol $\bar{x}$.
- Sample `svar` / `sstd` are the unbiased versions (`ddof=1`).
- `pdf` on a discrete distribution (or `pmf` on a continuous one) is an error — the message names the right method.
- `fit` exists only for the continuous distributions.
- `\mathrm{meanconf}` emits a small `mc_mean_ci` helper into the generating code — it wraps `st.t.interval` around the sample mean.
- `\mathrm{ttestind}` (independent) vs `\mathrm{ttestrel}` (paired) mirror scipy's `ttest_ind` / `ttest_rel`; `\mathrm{ttest}` stays the one-sample test.
- `\mathrm{kstest}` / `\mathrm{anderson}` test against `'norm'` by default; a second arg names another scipy distribution — `\mathrm{kstest}(d,\mathrm{expon})`.
- `\mathrm{chisquare}`'s expected counts must sum to the observed total (scipy rescales nothing).
- `\mathrm{anderson}` and `\mathrm{binomtest}` print their full result objects rather than a plain tuple.
- `\mathrm{bartlett}` float-coerces its samples in the emitted code — scipy raises on integer-typed data.
- Results are scipy result objects — the `statistic` / `pvalue` fields print as a pair; a `NaN` pair from a too-small sample is scipy's own answer, not a compile error.

Output targets

Pick a target from the **Output** menu in the header.

LaTeX

Beside each cell is its serialized LaTeX; **Copy** puts it on the clipboard.

Calculator

Each cell evaluates through SymPy — a real computer-algebra system that runs in the page as WebAssembly. It takes a while to download and boot; until the engine is ready, results come from a lightweight JavaScript engine and are dimmed and marked *estimate*. The chip at the top shows engine status.

- Each statement in a cell gets a result row; `≈` adds a decimal approximation when one exists.
- Cells share one namespace, in worksheet order: a variable or function defined in a cell is usable in every cell below it, and editing a cell re-evaluates the ones that depend on it.

- `!` marks a statement that failed, `i` a note; hover for the message.
- **Show generating code** reveals the exact Python program that produced the results, with a copy button.
- The `import *` checkbox in the header (shared with the Python target) writes the program against `from sympy import *` — off, it uses `import sympy as sp` with `sp.`-qualified names.
- The **Settings** menu (calculator and Python targets) exposes display plumbing and animation timing — default values are fine.

Python

Beside each cell is the SymPy program MathCompile generates for it, syntax-highlighted. **Copy** grabs one cell's lines; **Copy script** grabs the whole worksheet as one runnable program. The `import *` checkbox controls the import style. Statements the compiler can't express appear as `!` / `i` notes under the code.

One rule to know: write `\text{def} f(x) = 2x` (type `def` in Smart mode) to define a function — the generated code is `def f(x): return 2*x`. A bare `f(x)` is read as multiplication, `f * x`.

Keyboard reference

Key	What it does
`Enter`	new line in the cell; accepts an open `\command`; adds a row in a matrix
`Shift+Enter`	new cell below
`Right`, `Tab`	step out of the current block
`Esc`	dismiss the autocomplete menu or picker; returns focus to the field
`Ctrl+Space`	open the symbol picker
`Ctrl+K` / `⌘K`	command palette
`Shift+Space`	new column inside a matrix
`↑`, `↓`	move between cells at field edges
`Home`, `End`	block edges (`Ctrl+Home` / `Ctrl+End` for the whole field)
`Backspace` on a blank cell	delete the cell

Saving

Every worksheet autosaves to your browser's local storage — there's nothing to click. The header shows the active worksheet's name; its menu switches between sheets, creates new ones (**+ New worksheet**), renames (✎), and deletes (×, click twice to confirm).

Save to file downloads the active worksheet as a `.mathcompile.json` you can keep or share; **Open from file…** imports such a file back as a new worksheet. Clearing the site's data resets everything still in the browser.

Repository docs: `README.md` (build/run), `SYMBOLS.md` (command inventory), `AGENTS.md` (architecture), `CALCULATOR-ENGINES.md` (calculator internals).